

# Estruturas de Repetição em Java

Trilha Java Básico

Aula de 17 de agosto de 2026

## Sumário

|          |                                                                       |          |
|----------|-----------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introdução</b>                                                     | <b>2</b> |
| <b>2</b> | <b>Classificação das estruturas de repetição</b>                      | <b>2</b> |
| <b>3</b> | <b>Comando <code>for</code></b>                                       | <b>2</b> |
| 3.1      | Sintaxe . . . . .                                                     | 2        |
| 3.2      | Exemplo 1 — Contador de carneirinhos . . . . .                        | 3        |
| 3.2.1    | Simulação passo a passo (depuração) . . . . .                         | 3        |
| 3.3      | Exemplo 2 — <code>for</code> percorrendo um array . . . . .           | 4        |
| 3.4      | <code>for each</code> (“for it”) . . . . .                            | 4        |
| <b>4</b> | <b>Comandos <code>break</code> e <code>continue</code></b>            | <b>5</b> |
| <b>5</b> | <b>Comando <code>while</code></b>                                     | <b>6</b> |
| 5.1      | Sintaxe . . . . .                                                     | 6        |
| 5.2      | Exemplo 3 — A mesada do Joãozinho . . . . .                           | 6        |
| <b>6</b> | <b>Comando <code>do while</code></b>                                  | <b>8</b> |
| 6.1      | Sintaxe . . . . .                                                     | 8        |
| 6.2      | Exemplo 4 — O telefone tocando . . . . .                              | 8        |
| <b>7</b> | <b>Comparação: <code>while</code> <i>vs</i> <code>do while</code></b> | <b>9</b> |
| <b>8</b> | <b>Resumo e dicas de prática</b>                                      | <b>9</b> |

# 1 Introdução

Estruturas de repetição — também chamadas de *laços de repetição*, *laços de iteração* ou simplesmente *loops* — são comandos que permitem a **iteração de código**, ou seja, que os comandos presentes em um bloco sejam repetidos diversas vezes.

Por meio do *controle de fluxo de repetição*, as instruções podem acontecer recorrentemente (mais de uma vez) até que uma **condição** determine que não é mais necessária a continuação da execução daquele bloco.

## Observação

Os trechos de código abaixo foram reconstruídos a partir da narração da aula. Valores e nomes exibidos na tela (ex.: elementos de um *array*) podem ter pequenas variações em relação ao código original do professor, mas a lógica e a estrutura são fiéis ao que foi explicado.

## 2 Classificação das estruturas de repetição

Existem três estruturas de repetição em Java:

- **for** — tradução: “para”;
- **while** — tradução: “enquanto”;
- **do while** — tradução: “faça enquanto”.

Cada uma tem sua aplicabilidade, suas particularidades estruturais e dicas de uso, detalhadas a seguir.

## 3 Comando for

O comando **for** (“para”) permite que uma **variável contadora** seja testada e incrementada a cada iteração. A cada execução do fluxo, essa variável pode sofrer uma alteração de valor. O **for** recebe três informações, definidas na própria chamada do comando:

1. a **variável contadora** (inicialização);
2. a **condição** (expressão booleana de validação);
3. o **incremento** (atualização da variável).

### 3.1 Sintaxe

```
1 for (inicializacao; expressaoBooleana; incremento) {  
2     // corpo: comandos a serem repetidos  
3 }
```

Listing 1: Estrutura do for

As três partes ficam em uma única linha, separadas por ponto e vírgula (;):

- **Inicialização** — declara a variável contadora (ex.: `int i = 0`);
- **Expressão booleana** — validação; enquanto for **verdadeira**, o corpo é executado;
- **Incremento** — atualiza a variável a cada iteração (ex.: `i++`).

O corpo (bloco entre chaves) é executado inúmeras vezes, enquanto a expressão de validação for verdadeira. O comando deixa de executar quando a condição se torna **falsa**.

O fluxo é: *inicializa* → *testa a condição* → *executa o corpo* → *incrementa* → *testa de novo* ... até a condição se tornar falsa.

## 3.2 Exemplo 1 — Contador de carneirinhos

### Enunciado do Exemplo

Joãozinho precisa contar até 20 carneirinhos para pegar no sono. Monte um laço **for** com uma variável inteira (**int**) chamada **carneirinhos**, que inicia em 1 e, **enquanto for menor ou igual a 20**, é incrementada de 1 em 1 a cada iteração. A cada repetição, exiba a contagem atual. Ao final, depois que a condição deixar de ser verdadeira, exiba a mensagem “*Joãozinho dormiu!*”.

### Solução

```

1 public class ExemploFor {
2     public static void main(String[] args) {
3         for (int carneirinhos = 1; carneirinhos <= 20; carneirinhos
4             ++){
5             System.out.println("Contando carneirinhos: " +
6                 carneirinhos);
7         }
8         System.out.println("Joãozinho dormiu!");
9     }
10 }
```

Listing 2: ExemploFor.java

### 3.2.1 Simulação passo a passo (depuração)

Acompanhando a execução (por exemplo, com *breakpoint* na IDE):

1. **Inicialização:** `carneirinhos = 1`.
2. **Teste:** `1 <= 20` é *verdadeiro* → executa o corpo.
3. **Corpo:** imprime “*Contando carneirinhos: 1*”.
4. **Incremento:** `carneirinhos++` → `carneirinhos = 2`.
5. **Teste:** `2 <= 20` é *verdadeiro* → imprime 2.
6. ... e assim sucessivamente até imprimir 20.
7. Após imprimir 20, o incremento leva a `carneirinhos = 21`.

8. **Teste:**  $21 \leq 20$  é *falso*  $\rightarrow$  o laço é encerrado.
9. O programa segue normalmente para `System.out.println("Joãozinho dormiu!")`.

#### Observação

A saída imprime de 1 a 20 e, ao final, “*Joãozinho dormiu!*”. Isso confirma que o programa **continua executando** normalmente após o término do laço.

### 3.3 Exemplo 2 — for percorrendo um array

#### Enunciado do Exemplo

Crie um *array* de `String` com os nomes dos alunos e percorra-o com um laço **for**, imprimindo o aluno de cada índice. Use a propriedade `length` (tamanho) do *array* como limite da condição.

#### Solução

```
1 public class ExemploForArray {
2     public static void main(String[] args) {
3         String[] alunos = { "Felipe", "Jonas", "Julia", "Marcos" };
4
5         for (int x = 0; x < alunos.length; x++) {
6             System.out.println("O aluno no índice " + x + " é " +
7                 alunos[x]);
8         }
9     }
```

Listing 3: ExemploForArray.java

Explicação linha a linha:

- `String[] alunos = ...` — um *array* é um conjunto de elementos do mesmo tipo. Pode ser de `String`, `int`, `double` ou de qualquer classe (clientes, funcionários etc.). Cada elemento fica entre chaves, separado por vírgula.
- `for (int x = 0; x < alunos.length; x++)` — `x` representa o **índice** do elemento. O laço continua enquanto `x` for menor que o tamanho do *array*.
- `alunos[x]` — acessa o aluno que está na posição (índice) `x` da lista.

#### Observação

Em *arrays*, o índice dos elementos **inicia em 0** — por isso a variável `x` começa com 0. Se `alunos.length` vale 4, os índices válidos são 0, 1, 2 e 3.

### 3.4 for each (“for it”)

Para interagir com *arrays* e coleções de forma mais agradável, existe o **for each**, que percorre diretamente os elementos — sem a necessidade de controlar o índice manualmente:

```

1 String[] alunos = { "Felipe", "Jonas", "Julia", "Marcos" };
2
3 for (String aluno : alunos) {
4     System.out.println("O nome do aluno é: " + aluno);
5 }

```

Listing 4: Exemplo de for each

O uso do `for each` está fortemente relacionado a cenários com *arrays*/coleções. A cada iteração, a variável temporária `aluno` (do tipo do elemento, aqui `String`) recebe **automaticamente** o elemento atual da coleção. Os dois pontos (`:`) indicam “a cada elemento de”.

## 4 Comandos `break` e `continue`

Dentro de laços, duas palavras têm interação direta com o contexto de repetição:

- `break` — significa **quebrar / parar / interromper**. Interrompe **todo** o laço.
- `continue` — como o nome diz, **continua o laço**. Interrompe somente a **iteração atual** (pula para a próxima).

### Enunciado do Exemplo

Considere um laço `for` que conta de 1 a 5. Dentro dele existe uma condição: *se o número for igual a 3, interromper o fluxo*. Qual o resultado da execução? Em seguida, troque o `break` por `continue` e observe o que muda no resultado.

### Solução

Com `break`, imprime apenas **1 e 2**: ao chegar em 3, o laço é interrompido por completo — mesmo a condição determinando a contagem de 1 a 5.

Com `continue`, imprime **1, 2, 4 e 5**: ao chegar em 3, apenas aquela iteração é pulada; o laço segue normalmente para 4 e 5.

```

1 for (int numero = 1; numero <= 5; numero++) {
2     if (numero == 3)
3         break;
4     System.out.println(numero);
5 }
6 // Saída: 1, 2

```

Listing 5: Com `break`

```

1 for (int numero = 1; numero <= 5; numero++) {
2     if (numero == 3)
3         continue;
4     System.out.println(numero);
5 }
6 // Saída: 1, 2, 4, 5

```

Listing 6: Com `continue`

### Observação

O **break** é útil quando uma **regra de negócio**, ao se tornar verdadeira, deve interromper toda a execução. O **continue** é útil quando se quer apenas *desconsiderar* uma determinada iteração, sem parar o laço.

## 5 Comando while

O laço **while** (“enquanto”) determina que, **enquanto uma condição for válida, o bloco de código será executado**. Ele **testa a condição antes de executar o código**: se a condição já for inválida no primeiro teste, o bloco **nem chega a ser executado**.

### 5.1 Sintaxe

```
1 while (expressaoBooleana) {  
2     // corpo: comandos executados enquanto a condição for verdadeira  
3 }
```

Listing 7: Estrutura do while

O comando será executado até que a expressão de validação se torne **falsa**.

### Observação

É preciso garantir que, em algum momento, a condição se torne **falsa**. Caso contrário, cria-se um **laço infinito** (o bloco nunca para de executar).

### 5.2 Exemplo 3 — A mesada do Joãozinho

#### Enunciado do Exemplo

Joãozinho recebeu **R\$ 50,00** de mesada e foi a uma loja de doces para gastar todo o dinheiro. Enquanto o valor da mesada for **maior que zero**, ele adiciona ao carrinho um doce de valor aleatório (entre R\$ 2 e R\$ 8) e desconta esse valor da mesada. Ao final, exiba o valor restante da mesada e uma mensagem informando que Joãozinho gastou toda a sua mesada em doces.

## Solução

```
1 import java.util.concurrent.ThreadLocalRandom;
2
3 public class ExemploWhile {
4     public static void main(String[] args) {
5         double mesada = 50.0;
6
7         while (mesada > 0) {
8             double valorDoce = valorAleatorio();
9             if (valorDoce > mesada) {
10                 valorDoce = mesada;    // adaptação: evita mesada
                                     negativa
11             }
12             System.out.println("Doce do valor: " + valorDoce + "
                                adicionado no carrinho");
13             mesada = mesada - valorDoce;
14         }
15
16         System.out.println("Mesada: " + mesada);
17         System.out.println("Joãozinho gastou toda a sua mesada em
                                doces");
18     }
19
20     private static double valorAleatorio() {
21         return ThreadLocalRandom.current().nextDouble(2, 8);
22     }
23 }
```

Listing 8: ExemploWhile.java

Explicação:

- `double mesada = 50.0;` — a mesada inicia com 50.
- `while (mesada > 0)` — enquanto a mesada for maior que zero, o laço executa: pega um doce de valor aleatório e subtrai da mesada.
- `valorAleatorio()` — método auxiliar que retorna um `double` aleatório entre 2 e 8, usando `ThreadLocalRandom`. (Não é requisito do `while`; serve apenas para simular valores aleatórios.)
- `if (valorDoce > mesada) valorDoce = mesada;` — adaptação para que a mesada não fique negativa (regra de negócio). Sem esse trecho o código também funciona, mas poderia terminar com valor negativo.

## Observação

A quantidade de iterações não é fixa: depende dos valores aleatórios sorteados. O laço executa enquanto a condição for verdadeira — neste exemplo, até a mesada chegar a zero (ou próximo disso) e a condição `mesada > 0` se tornar falsa.

## 6 Comando do while

O `do while` (“faça enquanto”) tem a mesma proposta do `while` (executa enquanto a condição for válida), mas com uma diferença fundamental: **testa a condição depois de executar o código**, garantindo que o bloco seja executado **pelo menos uma vez**.

Assim, mesmo que a condição já seja inválida no primeiro teste, o bloco executa uma vez antes de parar.

### 6.1 Sintaxe

```
1 do {  
2     // corpo: executado ao menos uma vez  
3 } while (expressaoBooleana);
```

Listing 9: Estrutura do `do while`

Note o ponto e vírgula (;) ao final da linha do `while` — obrigatório nessa estrutura.

### 6.2 Exemplo 4 — O telefone tocando

#### Enunciado do Exemplo

Joãozinho ligou para um amigo. Enquanto o telefone estiver **tocando**, ele aguarda; quando o amigo **atender**, o fluxo é interrompido e parte-se para a conversa (“Alô!”). Use um `do while` para simular a chamada: o telefone deve “tocar” ao menos uma vez antes de verificar se o amigo atendeu.

#### Solução

```
1 import java.util.Random;  
2  
3 public class ExemploDoWhile {  
4     public static void main(String[] args) {  
5         System.out.println("Discando...");  
6  
7         do {  
8             System.out.println("Telefone tocando");  
9         } while (tocando());  
10  
11         System.out.println("Alô!!!");  
12     }  
13  
14     private static boolean tocando() {  
15         boolean atendeu = new Random().nextInt(3) + 1 == 1;    //  
16             valor aleatório entre 1 e 3  
17         System.out.println("Atendeu? " + atendeu);  
18         return !atendeu;    // continua "tocando" enquanto NÃO  
19             atendeu  
20     }  
21 }
```

Listing 10: ExemploDoWhile.java

Explicação:



- `do ... while (tocando());` — o corpo executa ao menos uma vez (“Telefone tocando”) e, só depois, a condição é testada.
- `tocando()` — método auxiliar que retorna `boolean` (*sim ou não*: atendeu ou não atendeu). Gera um valor aleatório entre 1 e 3; se for igual a 1, significa que **atendeu**.
- `return !atendeu;` — usa o operador unário de negação (`!`, “exclamação”). Se atendeu, nega para `false` e o laço para de “tocar”; senão, retorna `true` e o telefone continua tocando.

### Observação

Por ser aleatório, o número de “toques” muda a cada execução: às vezes o amigo atende de primeira; outras vezes o telefone toca várias vezes. O que é garantido é que o telefone **toca ao menos uma vez** antes da verificação — o que não aconteceria com o `while`, que testa a condição antes.

## 7 Comparação: `while` vs `do while`

| <code>while</code>                                                                   | <code>do while</code>                                                          |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Testa a condição <b>antes</b> de executar.                                           | Testa a condição <b>depois</b> de executar.                                    |
| Pode <b>não executar nenhuma vez</b> (se a condição já for falsa no primeiro teste). | Executa <b>pelo menos uma vez</b> , mesmo que a condição seja falsa de início. |

## 8 Resumo e dicas de prática

- `for` — ideal quando se conhece (ou se controla) a quantidade de repetições por meio de um contador.
- `while` — ideal quando a repetição depende de uma condição que é validada antes de cada execução.
- `do while` — ideal quando o bloco **precisa** executar ao menos uma vez antes da validação.
- `break` interrompe todo o laço; `continue` pula apenas a iteração atual.
- Em *arrays*, o índice começa em 0.

Para consolidar o conteúdo, o professor recomenda praticar de forma isolada, por exemplo:

- um contador de 0 a 100;
- um contador de números positivos;
- definir cenários onde `break` e `continue` se encaixam;
- criar um *array* e listar todas as frutas de uma cesta de frutas;
- usar a depuração (*breakpoint*) para acompanhar o valor das variáveis a cada iteração.